The Library Is the Oracle

Verification-first SoC generation from diagrams

Diagram → verified SoC: 2 model calls, 42 seconds —
30/30 self-test · KAT 79/79 (both oracles) · cycle-identical to reference

Harikrishnan KC · Team **Chip Convergence** · greatharikrishnan@gmail.com

NXP Problem · ICLAD-DAC 2026 GenAI Chip Hackathon

1 · The problem, and what actually scores

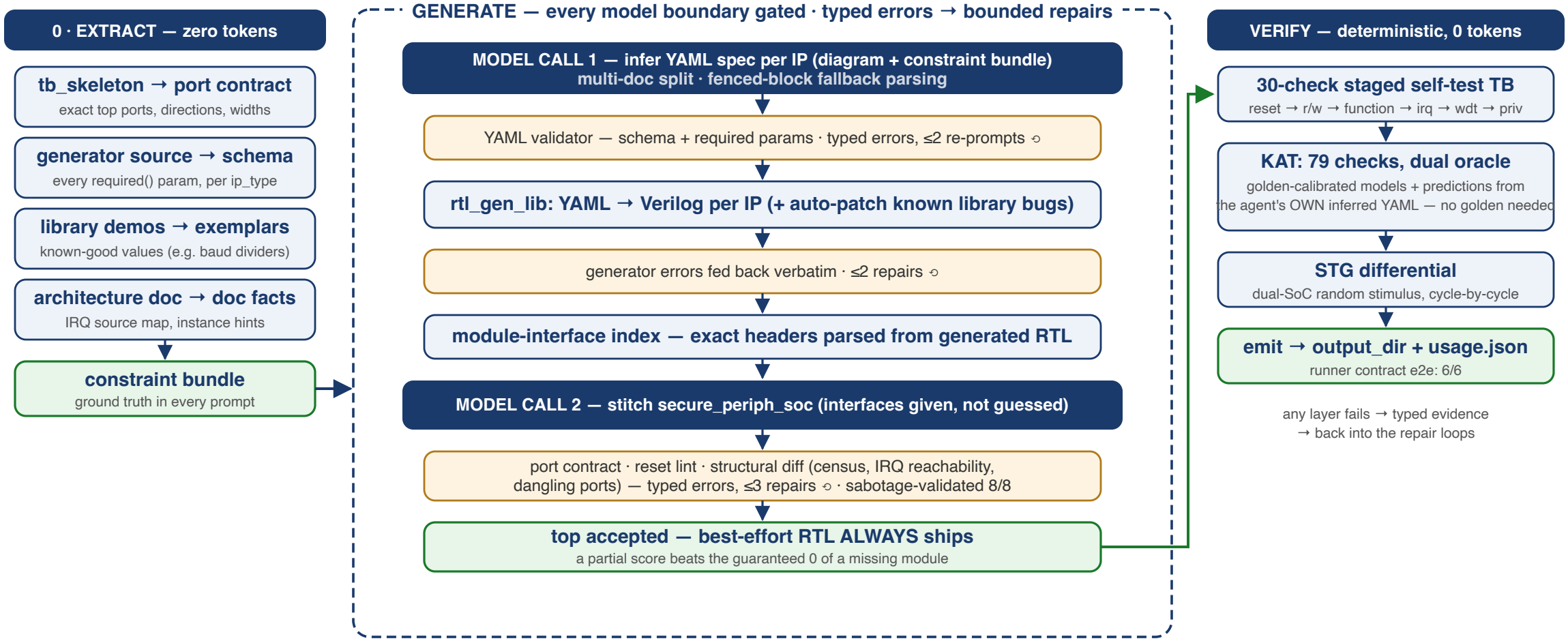
Given: one architecture HTML (diagrams + minimal spec), a TB skeleton fixing the top port contract, and `rtl_gen_lib` (YAML → Verilog generators). No YAML, no RTL, no text spec.

The agent must: read diagrams → infer YAML per IP → generate → stitch `secure_periph_soc` → verify.

Scoring reality:

Dimension	Rule	Consequence for agent design
Correctness	hidden golden TB, `passed/total`	**compile fail = 0** → never ship a broken top
Efficiency	total tokens, tiebreak	every re-prompt must earn its cost
Evaluation	runner invokes agent via `info.json` + HTTP model endpoint	contract compliance is table stakes

2 · What we do — one picture



Live: **2 model calls, 42 s** → 30/30 · KAT 79/79 (both oracles) · STG 3662 cycles, **0 differing** vs hand-built reference.

3 · Principles

1. **Tools before tokens** — the port contract, the YAML schema, the known-good exemplars and the doc facts are *extracted deterministically* from the provided materials and injected as ground truth; the model reads diagrams, not tea leaves.
2. **Constraints enforced by tooling, not prompting** — published evidence (SLDB, SpecAssess): frontier models alter ports they were told not to touch; reset semantics is the #1 silently-misspecified element. Every model boundary gets a deterministic gate.
3. **Every failure is a typed error** — and typed errors are repair fuel: bounded, targeted re-prompts with the exact violation, never "try again".
4. **Verify without a golden** — the oracle predicts from the agent's *own declared design*, so it works on hidden testcases where no reference exists.
5. **Always ship** — best-effort RTL beats the guaranteed zero of a missing module.

4 · Tools before tokens: what's extracted, zero-token

Extracted (deterministically, at runtime)	From	Feeds
Exact top-port contract (names, dirs, widths)	`tb_skeleton`	stitch prompt + hard gate
Required-params schema, per ip_type	generator *source* (`required()` calls)	spec prompt + YAML validator
Known-good exemplar values	library demo specs	spec prompt (e.g. baud divider)
IRQ source map, instance hints	architecture doc text	spec + stitch prompts
Module-interface index (exact headers)	the just-generated RTL	stitch prompt — ports *given*, not guessed

All of it re-derives itself on **any** testcase — nothing is hardcoded to the easy SoC.
The live proof: `default_div = 26` appears **nowhere in the doc**; it was recovered from library demo exemplars (115200 baud @ 50 MHz) — the library's implicit knowledge, mined.

5 · The correctness firewall (all sabotage-validated)

Gate	Catches	Validated
`port_contract`	renamed / missing / wrong-width top ports vs skeleton	typed errors on all fault classes
`reset_lint`	por_n feeding anything but the synchronizer; split reset nets	reset = the known killer
`structural_diff`	instance census, dangling ports, IRQ reachability, fabric hang-off, bridge bus	**8/8** sabotage suite
library fixups	known non-compiling generator output (auto-patch)	behavior-neutral

Example typed error, verbatim from a live run — this text *is* the repair prompt:

```
struct-irq: i_uart.irq net 'uart_irq' never reaches u_irq_aggregator.irq_src
```

6 · Verification depth: three independent layers

1. 30-check staged self-test TB — reset → r/w → function → irq → watchdog → privilege; first-failing stage localizes bugs for repair.

2. KAT engine — 125-command vector program, **79 known-answer checks**, replay-TB + Python comparison, two oracles.

3. STG differential — dual-SoC random-stimulus trace diff (LCG + burst schedules), cycle-by-cycle.

The layers are complementary — measured example: a FIFO_DEPTH 16→8 sabotage **passes the 30-check TB (30/30)** but the KAT status read catches it in 5 trace lines: `0x89 vs 0x88` — literally the `tx_full` bit.

7 · The earned asset: 20 library IPs, modeled bugs-included

The hidden golden TB is built on the **same generator library** — so our reference models are transcribed from the generated RTL, statement-for-statement, **quirks preserved** (a "first-principles correct" model would *fail* the golden TB — the library watchdog's kick genuinely never reloads, and our model asserts that actual behavior):

- cycle-stepped Python models for **all 20 library ip_types**
- easy-8 calibrated against golden-recorded traces: **0 mismatches**
- other 12 (AXI, TileLink, AES-128, FIFOs/SRAMs): **12/12 lockstep** vs library RTL, 2000 cycles each; AES bit-exact over ~150 encryptions

This suite is the NXP counterpart of a rule library: **earned by measurement, versioned, and reused by every layer of verification.**

8 · A no-golden oracle *mechanism* (a hidden-tier foundation)

KAT model path: replay vectors on the candidate → predict expected values from the reference models **parameterized by the agent's own inferred YAML** → compare.
No golden RTL, no golden TB required — the *mechanism* is topology-general.

Proof the oracle follows the declared design (not a hardcoded answer):

Candidate SoC	Model believes	Result
UART FIFO depth 8	`fifo_depth: 16`	FAIL 76/79 — catches the divergence
UART FIFO depth 8	`fifo_depth: 8` (plumbed)	PASS 79/79 — consistent design accepted

Honest scope: the mechanism generalizes; the *coverage* is the 8 easy IPs (0-mismatch calibrated) + 12 more library IPs modeled (slide 10) — a **foundation** for medium/hard, proven on the public case, not yet a topology-neutral solver. Runner contract: `python3 nxp_agent.py <info.json> --model <name>` — **6/6** e2e vs a mock endpoint.

9 · The result: perfect solve, 2 calls, 42 seconds

```
[1] model returned 8 YAML spec block(s)
[2] generated 8 IP file(s): ['sys_rst_sync.v', 'ahb_brg0.v', 'apb_fab0.v', 'uart0.v',
    'gpio0.v', 'timer0.v', 'wdt0.v', 'irq_agg0.v']
[3] wrote secure_periph_soc.v (contract clean, attempt 1)
[4] GATE: PASS — 30/30 PASS
[4b] KAT(golden): PASS — 79/79
[4b] KAT(model):  PASS — 79/79
[5] STG-DIFF: MATCH — 3662 cycles, 0 differing
    model calls: 2
```

The generated SoC is **cycle-identical** to our hand-built reference over 3662 random cycles — and the whole solve cost **2 model calls / ~40k tokens**.

10 · Medium/hard readiness

The library ships **20 generators; easy uses 8**. The other 12 are the likely medium/hard vocabulary — already modeled and validated:

- AXI-Lite: crossbar (2M×3S), SRAM, DMA engine — 12/12 lockstep
- TileLink: router, network interface
- AES-128 (bit-exact), FIFOs (sync/async gray-code), SRAMs, CDC, perf counter

Known library quirks catalogued (golden-TB-relevant): AES core **ignores its `encrypt` input**; TL router implements **no mesh routing** (all→local). Generator `MissingParameter` errors are typed and model-directed — purpose-built repair material.

11 · Economics & contest contributions

Item	Cost
Perfect easy-tier solve	**2 calls / ~40k tokens**
Repair bounds	≤2 spec re-prompts, ≤2 generator-repair, ≤3 stitch
Prevention > repair	schema in the *first* prompt ended the omission loop
Every verification gate	free — deterministic, local, zero tokens

3 toolkit bugs found & reported (details in the Learnings deck): `dma_engine` generates non-compiling RTL (hits *every* team; we auto-patch, behavior-neutral) · `apb_watchdog` kick never reloads (modeled as-is — oracle correctness) · NVIDIA aes flow masked failure.

12 · Next: what we build before DAC (agents updatable to Jul 26)

- **Testcase digest (designed)** — deterministic parse of the architecture doc's address-map tables + instance inventory, cross-checked against the model's specs; mismatches feed the existing repair loops. The per-testcase analog of our zero-token extraction — insurance for medium/hard doc complexity.
- **K-vote spec inference** — field-level majority across independent diagram reads; disagreement → targeted re-look (defense vs consistent misreads)
- **Best-of-M stitches** — KAT-model score as the selector
- **Model mixing on unlimited Vertex (Jul 19+)** — strong model for diagram reading, cheap model for repairs
- Medium/hard fixtures composed from the 12 modeled IPs; live-testcase runbook in repo

13 · Summary — every claim, one command

Claim	Reproduce with
Full pipeline offline	`python3 nxp_agent.py --model stub` → 30/30, KAT 79/79 ×2
Real-model solve	`python3 nxp_agent.py --model vertex --deep`
Gate sabotage suite	`python3 test_structural.py` → 8/8
Runner contract	`python3 test_runner_mode.py` → 6/6
12 IP models vs RTL	`python3 test_ip_models.py` → 12/12

Repo: github.com/chelsea85/chip-convergence-iclad26 (fresh-clone verified)

Companion deck: **Engineering Learnings** (attached) — what building this taught us.

Harikrishnan KC · Chip Convergence · greatharikrishnan@gmail.com